

webpack详解(1/4)

1. 课程目标

- 掌握webpack的运行机制；
- 掌握webpack的基本配置；

2. 课程大纲

1. webpack入门；
2. webpack配置；

3. webpack入门

github: <https://github.com/webpack/webpack>

官方地址: <https://webpack.js.org/>

3.1. 安装webpack

在用 Webpack 执行构建任务时需要通过 webpack 可执行文件去启动构建任务，所以需要安装 webpack 可执行文件。在安装 Webpack 前请确保你的系统安装了5.0.0及以上版本的 Node.js。

在开始给项目加入构建前，你需要先新建一个 Web 项目，方式包括：

- 新建一个目录，再进入项目根目录执行 `npm init` 来初始化最简单的采用了模块化开发的项目；
- 用脚手架工具 Yeoman 直接快速地生成一个最符合你的需求的项目；

3.1.1. 安装 Webpack 到本项目

要安装 Webpack 到本项目，可按照你的需要选择以下任意命令运行：

复制代码

```
1 # npm i -D 是 npm install --save-dev 的简写, 是指安装模块并保存到 package.json  
  的 devDependencies  
2 # 安装最新稳定版  
3 npm i -D webpack  
4  
5 # 安装指定版本  
6 npm i -D webpack@<version>  
7  
8 # 安装最新体验版本  
9 npm i -D webpack@beta
```

安装完后你可以通过这些途径运行安装到本项目的 Webpack:

- 在项目根目录下对应的命令行里通过 `node_modules/.bin/webpack` 运行 Webpack 可执行文件。
- 在 Npm Script 里定义的任务会优先使用本项目下的 Webpack, 代码如下:

复制代码

```
1 "scripts": {  
2   "start": "webpack --config webpack.config.js"  
3 }
```

3.1.2. 安装 Webpack 到全局

安装到全局后你可以在任何地方共用一个 Webpack 可执行文件, 而不用各个项目重复安装, 安装方式如下:

复制代码

```
1 npm i -g webpack
```

虽然介绍了以上两种安装方式, 但是我们推荐安装到本项目, 原因是可防止不同项目依赖不同版本的 Webpack 而导致冲突。

3.1.3. 使用 Webpack

下面通过 Webpack 构建一个采用 CommonJS 模块化编写的项目, 该项目有个网页会通过 JavaScript 在网页中显示 `Hello,Webpack`。

运行构建前, 先把要完成该功能的最基础的 JavaScript 文件和 HTML 建立好, 需要如下文件:

页面入口文件 index.html

复制代码

```
1 <html>
2 <head>
3   <meta charset="UTF-8">
4 </head>
5 <body>
6   <div id="app"></div>
7   <!--导入 Webpack 输出的 JavaScript 文件-->
8   <script src="./dist/bundle.js"></script>
9 </body>
10 </html>
```

JS 工具函数文件 show.js

复制代码

```
1 // 操作 DOM 元素, 把 content 显示到网页上
2 function show(content) {
3   window.document.getElementById('app').innerText = 'Hello,' + content;
4 }
5
6 // 通过 CommonJS 规范导出 show 函数
7 module.exports = show;
```

JS 执行入口文件 main.js

复制代码

```
1 // 通过 CommonJS 规范导入 show 函数
2 const show = require('./show.js');
3 // 执行 show 函数
4 show('Webpack');
```

Webpack 在执行构建时默认会从项目根目录下的 webpack.config.js 文件读取配置, 所以你还需新建它, 其内容如下:

```
1  const path = require('path');
2
3  module.exports = {
4    // JavaScript 执行入口文件
5    entry: './main.js',
6    output: {
7      // 把所有依赖的模块合并输出到一个 bundle.js 文件
8      filename: 'bundle.js',
9      // 输出文件都放到 dist 目录下
10     path: path.resolve(__dirname, './dist'),
11   }
12 };
```

由于 Webpack 构建运行在 Node.js 环境下，所以该文件最后需要通过 CommonJS 规范导出一个描述如何构建的 Object 对象。

此时项目目录如下：

```
1  |-- index.html
2  |-- main.js
3  |-- show.js
4  |-- webpack.config.js
```

一切文件就绪，在项目根目录下执行 webpack 命令运行 Webpack 构建，你会发现目录下多出一个 dist 目录，里面有个 bundle.js 文件，bundle.js 文件是一个可执行的 JavaScript 文件，它包含页面所依赖的两个模块 main.js 和 show.js 及内置的 webpackBootstrap 启动函数。这时你用浏览器打开 index.html 网页将会看到 Hello,Webpack。

Webpack 是一个打包模块化 JavaScript 的工具，它会从 main.js 出发，识别出源码中的模块化导入语句，递归的寻找出入口文件的所有依赖，把入口和其所有依赖打包到一个单独的文件中。从 Webpack2 开始，已经内置了对 ES6、CommonJS、AMD 模块化语句的支持。

3.2. 使用loader

基于上述代码，继续优化这个网页的 UI，为项目引入 CSS 代码让文字居中显示，main.css 的内容如下：


```

1  #app{
2    text-align: center;
3  }

```

Webpack 把一切文件看作模块，CSS 文件也不例外，要引入 `main.css` 需要像引入 JavaScript 文件那样，修改入口文件 `main.js` 如下：

```

1  // 通过 CommonJS 规范导入 CSS 模块
2  require('./main.css');
3  // 通过 CommonJS 规范导入 show 函数
4  const show = require('./show.js');
5  // 执行 show 函数
6  show('Webpack');

```

但是这样修改后去执行 Webpack 构建是会报错的，因为 Webpack 不原生支持解析 CSS 文件。要支持非 JavaScript 类型的文件，需要使用 Webpack 的 Loader 机制。Webpack 的配置修改使用如下：

```

1  const path = require('path');
2
3  module.exports = {
4    // JavaScript 执行入口文件
5    entry: './main.js',
6    output: {
7      // 把所有依赖的模块合并输出到一个 bundle.js 文件
8      filename: 'bundle.js',
9      // 输出文件都放到 dist 目录下
10     path: path.resolve(__dirname, './dist'),
11   },
12   module: {
13     rules: [
14       {
15         // 用正则去匹配要用该 loader 转换的 CSS 文件
16         test: /\.css$/,
17         use: ['style-loader', 'css-loader?minimize'],
18       }
19     ]
20   }
21 };

```

Loader 可以看作具有文件转换功能的翻译员，配置里的 `module.rules` 数组配置了一组规则，告诉 Webpack 在遇到哪些文件时使用哪些 Loader 去加载和转换。如上配置告诉 Webpack 在遇到以 `.css` 结尾的文件时先使用 `css-loader` 读取 CSS 文件，再交给 `style-loader` 把 CSS 内容注入到 JavaScript 里。在配置 Loader 时需要注意的是：

- `use` 属性的值需要是一个由 Loader 名称组成的数组，Loader 的执行顺序是由后到前的；
- 每一个 Loader 都可以通过 URL `querystring` 的方式传入参数，例如 `css-loader?minimize` 中的 `minimize` 告诉 `css-loader` 要开启 CSS 压缩；

在重新执行 Webpack 构建前要先安装新引入的 Loader：



复制代码

```
1 npm i -D style-loader css-loader
```

安装成功后重新执行构建时，你会发现 `bundle.js` 文件被更新了，里面注入了在 `main.css` 中写的 CSS，而不是会额外生成一个 CSS 文件。但是重新刷新 `index.html` 网页时将会发现 Hello,Webpack 居中 了，样式生效了！也许你会对此感到奇怪，第一次看到 CSS 被写在了 JavaScript 里！这其实都是 `style-loader` 的功劳，它的工作原理大概是把 CSS 内容用 JavaScript 里的字符串存储起来，在网页执行 JavaScript 时通过 DOM 操作动态地往 HTML head 标签里插入 HTML style 标签。

给 Loader 传入属性的方式除了有 `querystring` 外，还可以通过 Object 传入，以上的 Loader 配置可以修改为如下：



复制代码

```
1 use: [  
2   'style-loader',  
3   {  
4     loader: 'css-loader',  
5     options: {  
6       minimize: true,  
7     }  
8   }  
9 ]
```

除了在 `webpack.config.js` 配置文件中配置 Loader 外，还可以在源码中指定用什么 Loader 去处理文件。以加载 CSS 文件为例，修改上面例子中的 `main.js` 如下：



复制代码

```
1 require('style-loader!css-loader?minimize!./main.css');
```

这样就能指定对 `./main.css` 这个文件先采用 `css-loader` 再采用 `style-loader` 转换。

3.3. 使用plugin

Plugin 是用来扩展 Webpack 功能的，通过在构建流程里注入钩子实现，它给 Webpack 带来了很大的灵活性。

上面通过 Loader 加载了 CSS 文件，本节将通过 Plugin 把注入到 `bundle.js` 文件里的 CSS 提取到单独的文件中，配置修改如下：

 复制代码

```
1  const path = require('path');
2  const ExtractTextPlugin = require('extract-text-webpack-plugin');
3
4  module.exports = {
5    // JavaScript 执行入口文件
6    entry: './main.js',
7    output: {
8      // 把所有依赖的模块合并输出到一个 bundle.js 文件
9      filename: 'bundle.js',
10     // 把输出文件都放到 dist 目录下
11     path: path.resolve(__dirname, './dist'),
12   },
13   module: {
14     rules: [
15       {
16         // 用正则去匹配要用该 loader 转换的 CSS 文件
17         test: /\.css$/,
18         use: ExtractTextPlugin.extract({
19           // 转换 .css 文件需要使用的 Loader
20           use: ['css-loader'],
21         }),
22       }
23     ]
24   },
25   plugins: [
26     new ExtractTextPlugin({
27       // 从 .js 文件中提取出来的 .css 文件的名称
28       filename: `[name]_[contenthash:8].css`,
29     }),
30   ]
31 };
```

先安装：

```
1 npm i -D extract-text-webpack-plugin
```

安装成功后重新执行构建，你会发现 dist 目录下多出一个 main_XXX.css 文件，bundle.js 里也没有 CSS 代码了，再把该 CSS 文件引入到 index.html 里就完成了。

从以上代码可以看出，Webpack 是通过 plugins 属性来配置需要使用的插件列表的。plugins 属性是一个数组，里面的每一项都是插件的一个实例，在实例化一个组件时可以通过构造函数传入这个组件支持的配置属性。

例如 ExtractTextPlugin 插件的作用是提取出 JavaScript 代码里的 CSS 到一个单独的文件。对此你可以通过插件的 filename 属性，告诉插件输出的 CSS 文件名称是通过 [name]_[contenthash:8].css 字符串模版生成的，里面的 [name] 代表文件名称，[contenthash:8] 代表根据文件内容算出的8位 hash 值，还有很多配置选项可以在 [ExtractTextPlugin](#) 的主页上查到。

3.4. 使用Devserver

上面只是让 Webpack 正常运行起来了，但在实际开发中你可能会需要：

1. 提供 HTTP 服务而不是使用本地文件预览；
2. 监听文件的变化并自动刷新网页，做到实时预览；
3. 支持 Source Map ，以方便调试；

对于这些，Webpack 都为你考虑好了。Webpack 原生支持上述第2、3点内容，再结合官方提供的开发工具 [DevServer](#) 也可以很方便地做到第1点。DevServer 会启动一个 HTTP 服务器用于服务网页请求，同时会帮助启动 Webpack ，并接收 Webpack 发出的文件更变信号，通过 WebSocket 协议自动刷新网页做到实时预览。

下面继续集成 DevServer。首先需要安装 DevServer：

```
1 npm i -D webpack-dev-server
```

安装成功后执行 webpack-dev-server 命令，DevServer 就启动了，这时你会看到控制台有一串日志输出：

```
1 Project is running at http://localhost:8080/  
2 webpack output is served from /
```


这意味着 DevServer 启动的 HTTP 服务器监听在 `http://localhost:8080/`，DevServer 启动后会一直驻留在后台保持运行，访问这个网址你就能获取项目根目录下的 `index.html`。用浏览器打开这个地址你会发现页面空白错误原因是 `./dist/bundle.js` 加载404了。同时你会发现并没有文件输出到 `dist` 目录，原因是 DevServer 会把 Webpack 构建出的文件保存在内存中，在要访问输出的文件时，必须通过 HTTP 服务访问。由于 DevServer 不会理会 `webpack.config.js` 里配置的 `output.path` 属性，所以要获取 `bundle.js` 的正确 URL 是 `http://localhost:8080/bundle.js`，对应的 `index.html` 应该修改为：

 复制代码

```
1  <html>
2  <head>
3    <meta charset="UTF-8">
4  </head>
5  <body>
6    <div id="app"></div>
7    <!--导入 DevServer 输出的 JavaScript 文件-->
8    <script src="bundle.js"></script>
9  </body>
10 </html>
```

3.4.1. 实时预览

Webpack 在启动时可以开启监听模式，开启监听模式后 Webpack 会监听本地文件系统的变化，发生变化时重新构建出新的结果。Webpack 默认是关闭监听模式的，你可以在启动 Webpack 时通过 `webpack --watch` 来开启监听模式。

通过 DevServer 启动的 Webpack 会开启监听模式，当发生变化时重新执行完构建后通知 DevServer。DevServer 会让 Webpack 在构建出的 JavaScript 代码里注入一个代理客户端用于控制网页，网页和 DevServer 之间通过 WebSocket 协议通信，以方便 DevServer 主动向客户端发送命令。DevServer 在收到来自 Webpack 的文件变化通知时通过注入的客户端控制网页刷新。

接着上面的步骤，你可以试试修改 `main.js` `main.css` `show.js` 中的任何一个文件，保存后你会发现浏览器会被自动刷新，运行出修改后的效果。

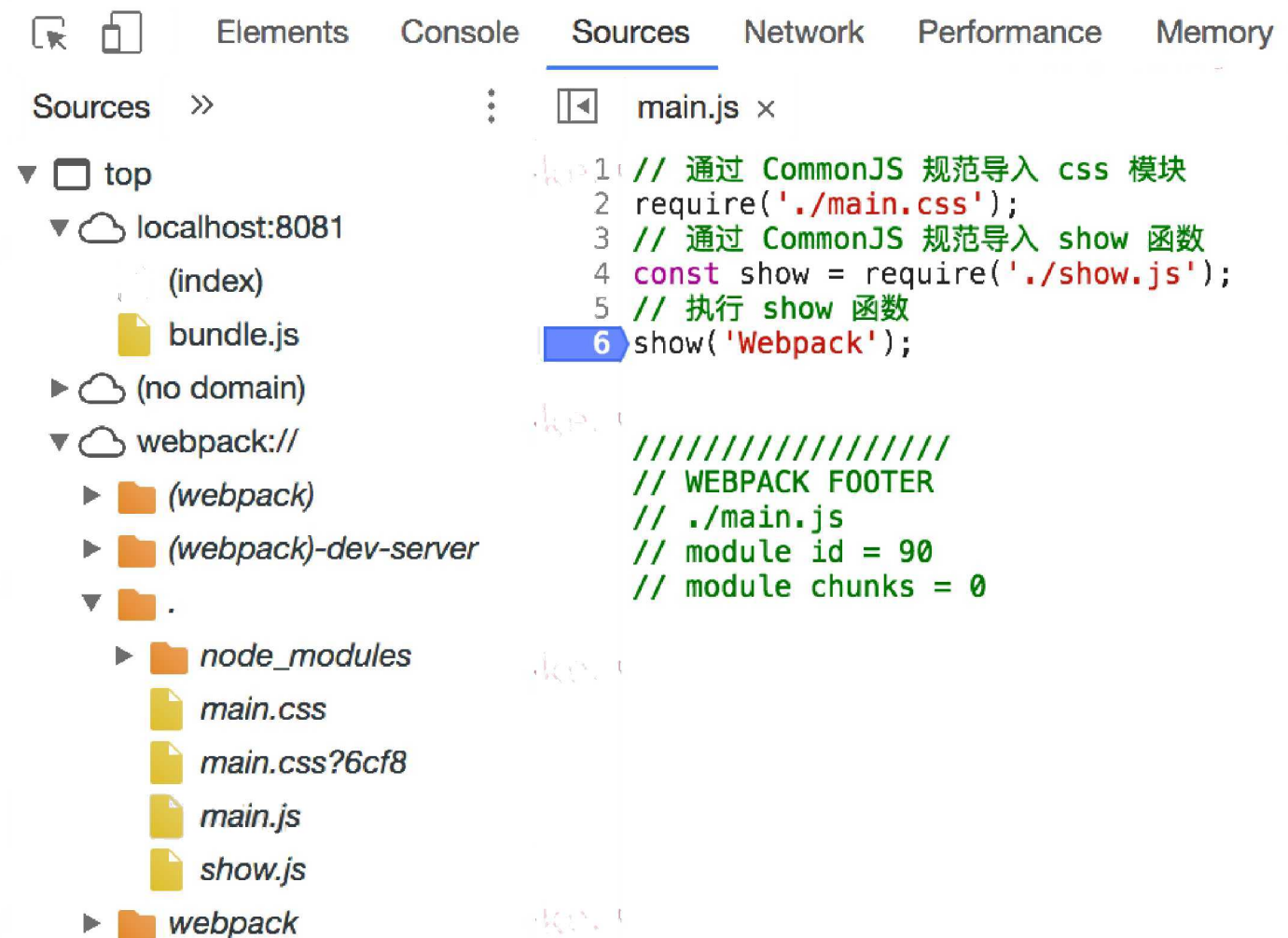
如果尝试修改 `index.html` 文件并保存，你会发现这并不会触发以上机制，导致这个问题的原因是 Webpack 在启动时会以配置里的 `entry` 为入口去递归解析出 `entry` 所依赖的文件，只有 `entry` 本身和依赖的文件才会被 Webpack 添加到监听列表里。而 `index.html` 文件是脱离了 JavaScript 模块化系统的，所以 Webpack 不知道它的存在。

3.4.2. 模块热替换

除了通过重新刷新整个网页来实现实时预览，DevServer 还有一种被称作模块热替换的刷新技术。模块热替换能做到在不重新加载整个网页的情况下，通过将被更新过的模块替换老的模块，再重新执行一次来实现实时预览。模块热替换相对于默认的刷新机制能提供更快的响应和更好的开发体验。模块热替换默认是关闭的，要开启模块热替换，你只需在启动 DevServer 时带上 `--hot` 参数，重启 DevServer 后再去更新文件就能体验到模块热替换的神奇了。

3.4.3. 支持Source Map

在浏览器中运行的 JavaScript 代码都是编译器输出的代码，这些代码的可读性很差。如果在开发过程中遇到一个不知道原因的 Bug，则你可能需要通过断点调试去找出问题。在编译器输出的代码上进行断点调试是一件辛苦和不优雅的事情，调试工具可以通过 Source Map 映射代码，让你在源代码上断点调试。Webpack 支持生成 Source Map，只需在启动时带上 `--devtool source-map` 参数。加上参数重启 DevServer 后刷新页面，再打开 Chrome 浏览器的开发者工具，就可在 Sources 栏中看到可调试的源代码了。



3.5. 核心概念

虽然Webpack 功能强大且配置项多，但只要你理解了其中的几个核心概念，就能随心应手地使用它。

Webpack 有以下几个核心概念：

1. Entry：入口，Webpack 执行构建的第一步将从 Entry 开始，可抽象成输入；
2. Module：模块，在 Webpack 里一切皆模块，一个模块对应着一个文件。Webpack 会从配置的 Entry 开始递归找出所有依赖的模块。；
3. Chunk：代码块，一个 Chunk 由多个模块组合而成，用于代码合并与分割；
4. Loader：模块转换器，用于把模块原内容按照需求转换成新内容；
5. Plugin：扩展插件，在 Webpack 构建流程中的特定时机注入扩展逻辑来改变构建结果或做你想要的事情；
6. Output：输出结果，在 Webpack 经过一系列处理并得出最终想要的代码后输出结果；

Webpack 启动后会从 Entry 里配置的 Module 开始递归解析 Entry 依赖的所有 Module 。每找到一个 Module ，就会根据配置的 Loader 去找出对应的转换规则，对 Module 进行转换后，再解析出当前 Module 依赖的 Module 。这些模块会以 Entry 为单位进行分组，一个 Entry 和其所有依赖的 Module 被分到一个组也就是一个 Chunk 。最后 Webpack 会把所有 Chunk 转换成文件输出。在整个流程中 Webpack 会在恰当的时机执行 Plugin 里定义的逻辑。

4. webpack配置

4.1. Entry

entry 是配置模块的入口，可抽象成输入，Webpack 执行构建的第一步将从入口开始搜寻及递归解析出所有入口依赖的模块。

entry 配置是必填的，若不填则将导致 Webpack 报错退出。

4.1.1. context

Webpack 在寻找相对路径的文件时会以 context 为根目录，context 默认为执行启动 Webpack 时所在的当前工作目录。如果想改变 context 的默认配置，则可以在配置文件里这样设置它：

```

1 module.exports = {
2   context: path.resolve(__dirname, 'app')
3 }

```

注意，`context` 必须是一个绝对路径的字符串。除此之外，还可以通过在启动 Webpack 时带上参数 `webpack --context` 来设置 `context`。

之所以在这里先介绍 `context`，是因为 `Entry` 的路径和其依赖的模块的路径可能采用相对于 `context` 的路径来描述，`context` 会影响到这些相对路径所指向的真实文件。

4.1.2. Entry类型

Entry 类型可以是以下三种中的一种或者相互组合：

类型	例子	含义
string	'./app/entry'	入口模块的文件路径，可以是相对路径。
array	['./app/entry1', './app/entry2']	入口模块的文件路径，可以是相对路径。
object	<pre> { a: './app/entry-a', b: ['./app/entry-b1', './app/entry-b2'] } </pre>	配置多个入口，每个入口生成一个 Chunk

如果是 array 类型，则搭配 `output.library` 配置项使用时，只有数组里的最后一个入口文件的模块会被导出。

4.1.3. Chunk名称

Webpack 会为每个生成的 Chunk 取一个名称，Chunk 的名称和 Entry 的配置有关：

- 如果 entry 是一个 string 或 array，就只会生成一个 Chunk，这时 Chunk 的名称是 main；

- 如果 entry 是一个 object，就可能会出现多个 Chunk，这时 Chunk 的名称是 object 键值对里键的名称；

4.1.4. 动态配置Entry

假如项目里有多个页面需要为每个页面的入口配置一个 Entry，但这些页面的数量可能会不断增长，则这时 Entry 的配置会受到其他因素的影响导致不能写成静态的值。其解决方法是把 Entry 设置成一个函数去动态返回上面所说的配置，代码如下：

 复制代码

```
1 // 同步函数
2 entry: () => {
3   return {
4     a: './pages/a',
5     b: './pages/b',
6   }
7 };
8 // 异步函数
9 entry: () => {
10  return new Promise((resolve)=>{
11    resolve({
12      a: './pages/a',
13      b: './pages/b',
14    });
15  });
16 };
```

4.2. Output

output 配置如何输出最终想要的代码。output 是一个 object，里面包含一系列配置项：

4.2.1. filename

output.filename 配置输出文件的名称，为string 类型。如果只有一个输出文件，则可以把它写成静态不变的：

 复制代码

```
1 filename: 'bundle.js'
```

但是在有多个 Chunk 要输出时，就需要借助模版和变量了。前面说到 Webpack 会为每个 Chunk 取一个名称，可以根据 Chunk 的名称来区分输出的文件名：

 复制代码

```
1 filename: '[name].js'
```

代码里的 `[name]` 代表用内置的 `name` 变量去替换 `[name]`，这时你可以把它看作一个字符串模块函数，每个要输出的 Chunk 都会通过这个函数去拼接出输出的文件名称。

内置变量除了 `name` 还包括：

变量名	含义
<code>id</code>	Chunk 的唯一标识，从0开始
<code>name</code>	Chunk 的名称
<code>hash</code>	Chunk 的唯一标识的 Hash 值
<code>chunkhash</code>	Chunk 内容的 Hash 值

其中 `hash` 和 `chunkhash` 的长度是可指定的，`[hash:8]` 代表取8位 Hash 值，默认是20位。

注意 [ExtractTextWebpackPlugin](#) 插件是使用 `contenthash` 来代表哈希值而不是 `chunkhash`，原因在于 `ExtractTextWebpackPlugin` 提取出来的内容是代码内容本身而不是由一组模块组成的 Chunk。

4.2.2. chunkFilename

`output.chunkFilename` 配置无入口的 Chunk 在输出时的文件名称。 `chunkFilename` 和上面的 `filename` 非常类似，但 `chunkFilename` 只用于指定在运行过程中生成的 Chunk 在输出时的文件名称。常见的会在运行时生成 Chunk 场景有在使用 `CommonChunkPlugin`、使用 `import('path/to/module')` 动态加载等时。 `chunkFilename` 支持和 `filename` 一致的内置变量。

4.2.3. path

`output.path` 配置输出文件存放在本地的目录，必须是 `string` 类型的绝对路径。通常通过 Node.js 的 `path` 模块去获取绝对路径：

 复制代码

```
1 path: path.resolve(__dirname, 'dist_[hash]')
```

4.2.4. publicPath

在复杂的项目里可能会有有一些构建出的资源需要异步加载，加载这些异步资源需要对应的 URL 地址。

`output.publicPath` 配置发布到线上资源的 URL 前缀，为 `string` 类型。默认值是空字符串 `"`，即使用相对路径。

这样说可能有点抽象，举个例子，需要把构建出的资源文件上传到 CDN 服务上，以利于加快页面的打开速度。配置代码如下：



复制代码

```
1 filename: '[name]_[chunkhash:8].js'
2 publicPath: 'https://cdn.example.com/assets/'
```

这时发布到线上的 HTML 在引入 JavaScript 文件时就需要：



复制代码

```
1 <script src='https://cdn.example.com/assets/a_12345678.js'></script>
```

使用该配置项时要小心，稍有不慎将导致资源加载404错误。

`output.path` 和 `output.publicPath` 都支持字符串模版，内置变量只有一个：`hash` 代表一次编译操作的 Hash 值。

4.2.5. crossOriginLoading

Webpack 输出的部分代码块可能需要异步加载，而异步加载是通过 [JSONP](#) 方式实现的。JSONP 的原理是动态地向 HTML 中插入一个 `<script src="url"></script>` 标签去加载异步资源。

`output.crossOriginLoading` 则是用于配置这个异步插入的标签的 `crossorigin` 值。

`script` 标签的 `crossorigin` 属性可以取以下值：

- `anonymous` (默认) 在加载此脚本资源时不会带上用户的 Cookies；
- `use-credentials` 在加载此脚本资源时会带上用户的 Cookies；

通常用设置 `crossorigin` 来获取异步加载的脚本执行时的详细错误信息。

4.2.6. libraryTarget 和 library

当用 Webpack 去构建一个可以被其他模块导入使用的库时需要用到它们。

- `output.libraryTarget` 配置以何种方式导出库；

- `output.library` 配置导出库的名称；

它们通常搭配在一起使用。

`output.libraryTarget` 是字符串的枚举类型，支持以下配置：

4.2.6.1. var (默认)

编写的库将通过 `var` 被赋值给通过 `library` 指定名称的变量。

假如配置了 `output.library='LibraryName'`，则输出和使用的代码如下：

```
1 // Webpack 输出的代码
2 var LibraryName = lib_code;
3
4 // 使用库的方法
5 LibraryName.doSomething();
```

 复制代码

假如 `output.library` 为空，则将直接输出：

```
1 lib_code
```

 复制代码

其中 `lib_code` 代指导出库的代码内容，是有返回值的一个自执行函数。

4.2.6.2. commonjs

编写的库将通过 CommonJS 规范导出。

假如配置了 `output.library='LibraryName'`，则输出和使用的代码如下：

```
1 // Webpack 输出的代码
2 exports['LibraryName'] = lib_code;
3
4 // 使用库的方法
5 require('library-name-in-npm')['LibraryName'].doSomething();
```

 复制代码

4.2.6.3. commonjs2

编写的库将通过 CommonJS2 规范导出，输出和使用的代码如下：

```
1 // Webpack 输出的代码
2 module.exports = lib_code;
3
4 // 使用库的方法
5 require('library-name-in-npm').doSomething();
```

CommonJS2 和 CommonJS 规范很相似，差别在于 CommonJS 只能用 exports 导出，而 CommonJS2 在 CommonJS 的基础上增加了 module.exports 的导出方式。

在 output.libraryTarget 为 commonjs2 时，配置 output.library 将没有意义。

4.2.6.4. this

编写的库将通过 this 被赋值给通过 library 指定的名称，输出和使用的代码如下：

```
1 // Webpack 输出的代码
2 this['LibraryName'] = lib_code;
3
4 // 使用库的方法
5 this.LibraryName.doSomething();
```

4.2.6.5. window

编写的库将通过 window 被赋值给通过 library 指定的名称，即把库挂载到 window 上，输出和使用的代码如下：

```
1 // Webpack 输出的代码
2 window['LibraryName'] = lib_code;
3
4 // 使用库的方法
5 window.LibraryName.doSomething();
```

4.2.6.6. global

编写的库将通过 global 被赋值给通过 library 指定的名称，即把库挂载到 global 上，输出和使用的代码如下：

```
1 // Webpack 输出的代码
2 global['LibraryName'] = lib_code;
3
4 // 使用库的方法
5 global.LibraryName.doSomething();
```

4.2.7. libraryExport

output.libraryExport 配置要导出的模块中哪些子模块需要被导出。它只有在 output.libraryTarget 被设置成 commonjs 或者 commonjs2 时使用才有意义。

假如要导出的模块源代码是：

```
1 export const a=1;
2 export default b=2;
```

现在你想让构建输出的代码只导出其中的 a，可以把 output.libraryExport 设置成 a，那么构建输出的代码和使用方法将变成如下：

```
1 // Webpack 输出的代码
2 module.exports = lib_code['a'];
3
4 // 使用库的方法
5 require('library-name-in-npm')===1;
```

以上都是基础的配置，详情请见[官网](#)

4.3. Module

module 配置如何处理模块。

4.3.1. 配置Loader

rules 配置模块的读取和解析规则，通常用来配置 Loader。其类型是一个数组，数组里每一项都描述了如何去处理部分文件。配置一项 rules 时大致通过以下方式：

1. 条件匹配：通过 `test`、`include`、`exclude` 三个配置项来命中 Loader 要应用规则的文件；
2. 应用规则：对选中后的文件通过 `use` 配置项来应用 Loader，可以只应用一个 Loader 或者按照从后往前的顺序应用一组 Loader，同时还可以分别给 Loader 传入参数；
3. 重置顺序：一组 Loader 的执行顺序默认是从右到左执行，通过 `enforce` 选项可以让其中一个 Loader 的执行顺序放到最前或者最后；

下面来通过一个例子来说明具体使用方法：

 复制代码

```
1  module: {
2    rules: [
3      {
4        // 命中 JavaScript 文件
5        test: /\.js$/,
6        // 用 babel-loader 转换 JavaScript 文件
7        // ?cacheDirectory 表示传给 babel-loader 的参数，用于缓存 babel 编译结果加
        快重新编译速度
8        use: ['babel-loader?cacheDirectory'],
9        // 只命中src目录里的js文件，加快 Webpack 搜索速度
10       include: path.resolve(__dirname, 'src')
11     },
12     {
13       // 命中 SCSS 文件
14       test: /\.scss$/,
15       // 使用一组 Loader 去处理 SCSS 文件。
16       // 处理顺序为从后到前，即先交给 sass-loader 处理，再把结果交给 css-loader 最
        后再给 style-loader。
17       use: ['style-loader', 'css-loader', 'sass-loader'],
18       // 排除 node_modules 目录下的文件
19       exclude: path.resolve(__dirname, 'node_modules'),
20     },
21     {
22       // 对非文本文件采用 file-loader 加载
23       test: /\.?(gif|png|jpe?g|eot|woff|ttf|svg|pdf)$/,
24       use: ['file-loader'],
25     },
26   ]
27 }
```

在 Loader 需要传入很多参数时，你还可以通过一个 Object 来描述，例如在上面的 `babel-loader` 配置中有如下代码：

```

1  use: [
2    {
3      loader: 'babel-loader',
4      options: {
5        cacheDirectory: true,
6      },
7      // enforce: 'post' 的含义是把该 Loader 的执行顺序放到最后
8      // enforce 的值还可以是 pre, 代表把 Loader 的执行顺序放到最前面
9      enforce: 'post'
10   },
11   // 省略其它 Loader
12 ]

```

上面的例子中 `test` `include` `exclude` 这三个命中文件的配置项只传入了一个字符串或正则，其实它们还支持数组类型，使用如下：

```

1  {
2    test: [
3      /\.jsx?$/,
4      /\.tsx?$/
5    ],
6    include: [
7      path.resolve(__dirname, 'src'),
8      path.resolve(__dirname, 'tests'),
9    ],
10   exclude: [
11     path.resolve(__dirname, 'node_modules'),
12     path.resolve(__dirname, 'bower_modules'),
13   ]
14 }

```

4.3.1.1. noParse

`noParse` 配置项可以让 Webpack 忽略对部分没采用模块化的文件的递归解析和处理，这样做的好处是能提高构建性能。原因是一些库例如 jQuery、ChartJS 它们庞大又没有采用模块化标准，让 Webpack 去解析这些文件耗时又没有意义。

`noParse` 是可选配置项，类型需要是 `RegExp`、`[RegExp]`、`function` 其中一个。

例如想要忽略掉 jQuery、ChartJS，可以使用如下代码：


```

1 // 使用正则表达式
2 noParse: /jquery|chartjs/
3
4 // 使用函数, 从 Webpack 3.0.0 开始支持
5 noParse: (content)=> {
6   // content 代表一个模块的文件路径
7   // 返回 true or false
8   return /jquery|chartjs/.test(content);
9 }

```

注意被忽略掉的文件里不应该包含 `import` 、 `require` 、 `define` 等模块化语句, 不然会导致构建出的代码中包含无法在浏览器环境下执行的模块化语句。

4.3.1.2. parser

因为 Webpack 是以模块化的 JavaScript 文件为入口, 所以内置了对模块化 JavaScript 的解析功能, 支持 AMD、CommonJS、SystemJS、ES6。 `parser` 属性可以更细粒度的配置哪些模块语法要解析哪些不解析, 和 `noParse` 配置项的区别在于 `parser` 可以精确到语法层面, 而 `noParse` 只能控制哪些文件不被解析。 `parser` 使用如下:

```

1 module: {
2   rules: [
3     {
4       test: /\.js$/,
5       use: ['babel-loader'],
6       parser: {
7         amd: false, // 禁用 AMD
8         commonjs: false, // 禁用 CommonJS
9         system: false, // 禁用 SystemJS
10        harmony: false, // 禁用 ES6 import/export
11        requireInclude: false, // 禁用 require.include
12        requireEnsure: false, // 禁用 require.ensure
13        requireContext: false, // 禁用 require.context
14        browserify: false, // 禁用 browserify
15        requireJs: false, // 禁用 requirejs
16      }
17    },
18  ]
19 }

```

4.4. Resolve

Webpack 在启动后会从配置的入口模块出发找出所有依赖的模块，Resolve 配置 Webpack 如何寻找模块所对应的文件。Webpack 内置 JavaScript 模块化语法解析功能，默认会采用模块化标准里约定好的规则去寻找，但你也可以根据自己的需要修改默认的规则。

4.4.1. alias

`resolve.alias` 配置项通过别名来把原导入路径映射成一个新的导入路径。例如使用以下配置：

 复制代码

```
1 // Webpack alias 配置
2 resolve:{
3   alias:{
4     components: './src/components/'
5   }
6 }
```

当你通过 `import Button from 'components/button'` 导入时，实际上被 alias 等价替换成了 `import Button from './src/components/button'`。

以上 alias 配置的含义是把导入语句里的 components 关键字替换成 `./src/components/`。

这样做可能会命中太多的导入语句，alias 还支持 `$` 符号来缩小范围到只命中以关键字结尾的导入语句：

 复制代码

```
1 resolve:{
2   alias:{
3     'react$': '/path/to/react.min.js'
4   }
5 }
```

`react$` 只会命中以 `react` 结尾的导入语句，即只会把 `import 'react'` 关键字替换成 `import '/path/to/react.min.js'`。

4.4.2. mainFields

有一些第三方模块会针对不同环境提供几分代码。例如分别提供采用 ES5 和 ES6 的2份代码，这2份代码的位置写在 `package.json` 文件里，如下：

```
1  {  
2    "jsnext:main": "es/index.js", // 采用 ES6 语法的代码入口文件  
3    "main": "lib/index.js" // 采用 ES5 语法的代码入口文件  
4  }
```

Webpack 会根据 mainFields 的配置去决定优先采用那份代码，mainFields 默认如下：

```
1  mainFields: ['browser', 'main']
```

Webpack 会按照数组里的顺序去package.json 文件里寻找，只会使用找到的第一个。

假如你想优先采用 ES6 的那份代码，可以这样配置：

```
1  mainFields: ['jsnext:main', 'browser', 'main']
```

4.4.3. extensions

在导入语句没带文件后缀时，Webpack 会自动带上后缀后去尝试访问文件是否存在。

resolve.extensions 用于配置在尝试过程中用到的后缀列表，默认是：

```
1  extensions: ['.js', '.json']
```

也就是说当遇到 require('./data') 这样的导入语句时，Webpack 会先去寻找 ./data.js 文件，如果该文件不存在就去寻找 ./data.json 文件，如果还是找不到就报错。

假如你想让 Webpack 优先使用目录下的 TypeScript 文件，可以这样配置：

```
1  extensions: ['.ts', '.js', '.json']
```

4.4.4. modules

resolve.modules 配置 Webpack 去哪些目录下寻找第三方模块，默认是只会去 node_modules 目录下寻找。有时你的项目里会有一些模块会大量被其它模块依赖和导入，由于其它模块的位置分布不定，

针对不同的文件都要去计算被导入模块文件的相对路径，这个路径有时候会很长，就像这样 `import` `'../../../components/button'` 这时你可以利用 `modules` 配置项优化，假如那些被大量导入的模块都在 `./src/components` 目录下，把 `modules` 配置成

 复制代码

```
1 modules: ['./src/components', 'node_modules']
```

后，你可以简单通过 `import 'button'` 导入。

4.4.5. descriptionFiles

`resolve.descriptionFiles` 配置描述第三方模块的文件名称，也就是 `package.json` 文件。默认如下：

 复制代码

```
1 descriptionFiles: ['package.json']
```

4.4.6. enforceExtension

`resolve.enforceExtension` 如果配置为 `true` 所有导入语句都必须带文件后缀，例如开启前 `import` `'./foo'` 能正常工作，开启后就必须写成 `import` `'./foo.js'`。

4.4.7. enforceModuleExtension

`enforceModuleExtension` 和 `enforceExtension` 作用类似，但 `enforceModuleExtension` 只对 `node_modules` 下的模块生效。`enforceModuleExtension` 通常搭配 `enforceExtension` 使用，在 `enforceExtension:true` 时，因为安装的第三方模块中大多数导入语句没带文件后缀，所以这时通过配置 `enforceModuleExtension:false` 来兼容第三方模块。

4.5. Plugin

Plugin 用于扩展 Webpack 功能，各种各样的 Plugin 几乎让 Webpack 可以做任何构建相关的事情。

4.5.1. 配置 Plugin

Plugin 的配置很简单，`plugins` 配置项接受一个数组，数组里每一项都是一个要使用的 Plugin 的实例，Plugin 需要的参数通过构造函数传入。

```
1  const CommonsChunkPlugin = require('webpack/lib/optimize/CommonsChunkPlugin');
2
3  module.exports = {
4    plugins: [
5      // 所有页面都会用到的公共代码提取到 common 代码块中
6      new CommonsChunkPlugin({
7        name: 'common',
8        chunks: ['a', 'b']
9      }),
10   ]
11  };
```

使用 Plugin 的难点在于掌握 Plugin 本身提供的配置项，而不是如何在 Webpack 中接入 Plugin。

几乎所有 Webpack 无法直接实现的功能都能在社区找到开源的 Plugin 去解决，你需要善于使用搜索引擎去寻找解决问题的方法。

4.6. DevServer

DevServer提供了一些配置项可以改变 DevServer 的默认行为。要配置 DevServer，除了在配置文件里通过 devServer 传入参数外，还可以通过命令行参数传入。注意只有在通过 DevServer 去启动 Webpack 时配置文件里 devServer 才会生效，因为这些参数所对应的功能都是 DevServer 提供的，Webpack 本身并不认识 devServer 配置项。

4.6.1. hot

devServer.hot 配置是否启用模块热替换功能。DevServer 默认的行为是在发现源代码被更新后会通过自动刷新整个页面来做到实时预览，开启模块热替换功能后将在不刷新整个页面的情况下通过用新模块替换老模块来做到实时预览。

4.6.2. inline

DevServer 的实时预览功能依赖一个注入到页面里的代理客户端去接受来自 DevServer 的命令和负责刷新网页的工作。devServer.inline 用于配置是否自动注入这个代理客户端到将运行在页面里的 Chunk 里去，默认是会自动注入。DevServer 会根据你是否开启 inline 来调整它的自动刷新策略：

- 如果开启 inline，DevServer 会在构建完变化后的代码时通过代理客户端控制网页刷新；
- 如果关闭 inline，DevServer 将无法直接控制要开发的网页。这时它会通过 iframe 的方式去运行要

开发的网页，当构建完变化后的代码时通过刷新 `iframe` 来实现实时预览。但这时你需要去 `http://localhost:8080/webpack-dev-server/` 实时预览你的网页了；

如果你想使用 `DevServer` 去自动刷新网页实现实时预览，最方便的方法是直接开启 `inline`。

4.6.3. historyApiFallback

`devServer.historyApiFallback` 用于方便的开发使用了 [HTML5 History API](#) 的单页应用。这类单页应用要求服务器在针对任何命中的路由时都返回一个对应的 HTML 文件，例如在访问 `http://localhost/user` 和 `http://localhost/home` 时都返回 `index.html` 文件，浏览器端的 JavaScript 代码会从 URL 里解析出当前页面的状态，显示出对应的界面。

配置 `historyApiFallback` 最简单的做法是：

```
1 historyApiFallback: true
```

 复制代码

这会导致任何请求都会返回 `index.html` 文件，这只能用于只有一个 HTML 文件的应用。

如果你的应用由多个单页应用组成，这就需要 `DevServer` 根据不同的请求来返回不同的 HTML 文件，配置如下：

```
1 historyApiFallback: {
2   // 使用正则匹配命中路由
3   rewrites: [
4     // /user 开头的都返回 user.html
5     { from: /^\/user/, to: '/user.html' },
6     { from: /^\/game/, to: '/game.html' },
7     // 其它的都返回 index.html
8     { from: /.\/, to: '/index.html' },
9   ]
10 }
```

 复制代码

4.6.4. contentBase

`devServer.contentBase` 配置 `DevServer` HTTP 服务器的文件根目录。默认情况下为当前执行目录，通常是项目根目录，所有一般情况下你不必设置它，除非你有额外的文件需要被 `DevServer` 服务。例如你想把项目根目录下的 `public` 目录设置成 `DevServer` 服务器的文件根目录，你可以这样配置：

```
1 devServer:{
2   contentBase: path.join(__dirname, 'public')
3 }
```

这里需要指出可能会让你疑惑的地方，DevServer 服务器通过 HTTP 服务暴露出的文件分为两类：

- 暴露本地文件；
- 暴露 Webpack 构建出的结果，由于构建出的结果交给了 DevServer，所以你在使用了 DevServer 时在本地找不到构建出的文件；

contentBase 只能用来配置暴露本地文件的规则，你可以通过 contentBase:false 来关闭暴露本地文件。

4.6.5. headers

devServer.headers 配置项可以在 HTTP 响应中注入一些 HTTP 响应头，使用如下：

```
1 devServer:{
2   headers: {
3     'X-foo': 'bar'
4   }
5 }
```

4.6.6. host

devServer.host 配置项用于配置 DevServer 服务监听的地址。例如你想要局域网中的其它设备访问你本地的服务，可以在启动 DevServer 时带上 `--host 0.0.0.0`。host 的默认值是 `127.0.0.1`

即只有本地可以访问 DevServer 的 HTTP 服务；

4.6.7. port

devServer.port 配置项用于配置 DevServer 服务监听的端口，默认使用 8080 端口。如果 8080 端口已经被其它程序占有就使用 8081，如果 8081 还是被占用就使用 8082，以此类推。

4.6.8. allowHosts

`devServer.allowedHosts` 配置一个白名单列表，只有 HTTP 请求的 HOST 在列表里才正常返回，使用如下：

 复制代码

```
1  allowedHosts: [  
2    // 匹配单个域名  
3    'host.com',  
4    'sub.host.com',  
5    // host2.com 和所有的子域名 *.host2.com 都将匹配  
6    '*.host2.com'  
7  ]
```

4.6.9. disableHostCheck

`devServer.disableHostCheck` 配置项用于配置是否关闭用于 DNS 重绑定的 HTTP 请求的 HOST 检查。DevServer 默认只接受来自本地的请求，关闭后可以接受来自任何 HOST 的请求。它通常用于搭配 `--host 0.0.0.0` 使用，因为你想要其它设备访问你本地的服务，但访问时是直接通过 IP 地址访问而不是 HOST 访问，所以需要关闭 HOST 检查。

4.6.10. https

DevServer 默认使用 HTTP 协议服务，它也能通过 HTTPS 协议服务。有些情况下你必须使用 HTTPS，例如 HTTP2 和 Service Worker 就必须运行在 HTTPS 之上。要切换成 HTTPS 服务，最简单的方式是：

 复制代码

```
1  devServer:{  
2    https: true  
3  }
```

DevServer 会自动的为你生成一份 HTTPS 证书。

如果你想用自己的证书可以这样配置：

```
1 devServer:{  
2   https: {  
3     key: fs.readFileSync('path/to/server.key'),  
4     cert: fs.readFileSync('path/to/server.crt'),  
5     ca: fs.readFileSync('path/to/ca.pem')  
6   }  
7 }
```

4.6.11. clientLogLevel

`devServer.clientLogLevel` 配置在客户端的日志等级，这会影响到你在浏览器开发者工具控制台里看到的日志内容。`clientLogLevel` 是枚举类型，可取如下之一的值 `none` | `error` | `warning` | `info` 。默认为 `info` 级别，即输出所有类型的日志，设置成 `none` 可以不输出任何日志；

4.6.12. compress

`devServer.compress` 配置是否启用 `gzip` 压缩。`boolean` 为类型，默认为 `false`；

4.6.13. open

`devServer.open` 用于在 `DevServer` 启动且第一次构建完时自动用你系统上默认的浏览器去打开要开发的网页。同时还提供 `devServer.openPage` 配置项用于打开指定 `URL` 的网页；

4.7. 其他配置

4.7.1. Target

JavaScript 的应用场景越来越多，从浏览器到 `Node.js`，这些运行在不同环境的 JavaScript 代码存在一些差异。`target` 配置项可以让 `Webpack` 构建出针对不同运行环境的代码。`target` 可以是以下之一：

target值	描述
web	针对浏览器 (默认), 所有代码都集中在一个文件里
node	针对 Node.js, 使用 require 语句加载 Chunk 代码
async-node	针对 Node.js, 异步加载 Chunk 代码
webworker	针对 WebWorker
electron-main	针对 Electron 主线程
electron-renderer	针对 Electron 渲染线程

例如当你设置 `target:'node'` 时, 源代码中导入 Node.js 原生模块的语句 `require('fs')` 将会被保留, `fs` 模块的内容不会打包进 Chunk 里。

4.7.2. Devtool

devtool 配置 Webpack 如何生成 Source Map, 默认值是 `false` 即不生成 Source Map, 想为构建出的代码生成 Source Map 以方便调试, 可以这样配置:

```
1 module.export = {
2   devtool: 'source-map'
3 }
```

复制代码

4.7.3. Watch 和 WatchOptions

webpack支持监听文件更新, 在文件发生变化时重新编译。在使用 Webpack 时监听模式默认是关闭的, 想打开需要如下配置:

```
1 module.export = {
2   watch: true
3 }
```

复制代码

在使用 DevServer 时，监听模式默认是开启的。

除此之外，Webpack 还提供了 watchOptions 配置项去更灵活的控制监听模式，使用如下

 复制代码

```
1  module.export = {
2    // 只有在开启监听模式时，watchOptions 才有意义
3    // 默认为 false，也就是不开启
4    watch: true,
5    // 监听模式运行时的参数
6    // 在开启监听模式时，才有意义
7    watchOptions: {
8      // 不监听的文件或文件夹，支持正则匹配
9      // 默认为空
10     ignored: /node_modules/,
11     // 监听到变化发生后会等300ms再去执行动作，防止文件更新太快导致重新编译频率太高
12     // 默认为 300ms
13     aggregateTimeout: 300,
14     // 判断文件是否发生变化是通过不停的去询问系统指定文件有没有变化实现的
15     // 默认每隔1000毫秒询问一次
16     poll: 1000
17   }
18 }
```

4.7.4. externals

Externals 用来告诉 Webpack 要构建的代码中使用了哪些不用被打包的模块，也就是说这些模版是外部环境提供的，Webpack 在打包时可以忽略它们。

有些 JavaScript 运行环境可能内置了一些全局变量或者模块，例如在你的 HTML HEAD 标签里通过以下代码：

 复制代码

```
1  <script src="path/to/jquery.js"></script>
```

引入 jQuery 后，全局变量 jQuery 就会被注入到网页的 JavaScript 运行环境里。

如果想在使用模块化的源代码里导入和使用 jQuery，可能需要这样：

 复制代码

```
1  import $ from 'jquery';
2  $('.my-element');
```

构建后你会发现输出的 Chunk 里包含的 jQuery 库的内容，这导致 jQuery 库出现了2次，浪费加载流量，最好是 Chunk 里不会包含 jQuery 库的内容。

Externals 配置项就是为了解决这个问题。

通过 externals 可以告诉 Webpack JavaScript 运行环境已经内置了那些全局变量，针对这些全局变量不用打包进代码中而是直接使用全局变量。要解决以上问题，可以这样配置 externals：

 复制代码

```
1 module.export = {
2   externals: {
3     // 把导入语句里的 jquery 替换成运行环境里的全局变量 jQuery
4     jquery: 'jQuery'
5   }
6 }
```

4.7.5. ResolveLoader

ResolveLoader 用来告诉 Webpack 如何去寻找 Loader，因为在使用 Loader 时是通过其包名称去引用的，Webpack 需要根据配置的 Loader 包名去找到 Loader 的实际代码，以调用 Loader 去处理源文件。

ResolveLoader 的默认配置如下：

 复制代码

```
1 module.exports = {
2   resolveLoader:{
3     // 去哪个目录下寻找 Loader
4     modules: ['node_modules'],
5     // 入口文件的后缀
6     extensions: ['.js', '.json'],
7     // 指明入口文件位置的字段
8     mainFields: ['loader', 'main']
9   }
10 }
```

该配置项常用于加载本地的 Loader。

4.8. 整体配置结构

```

1  const path = require('path');
2
3  module.exports = {
4    // entry 表示 入口, Webpack 执行构建的第一步将从 Entry 开始, 可抽象成输入。
5    // 类型可以是 string | object | array
6    entry: './app/entry', // 只有1个入口, 入口只有1个文件
7    entry: ['./app/entry1', './app/entry2'], // 只有1个入口, 入口有2个文件
8    entry: { // 有2个入口
9      a: './app/entry-a',
10     b: ['./app/entry-b1', './app/entry-b2']
11   },
12
13   // 如何输出结果: 在 Webpack 经过一系列处理后, 如何输出最终想要的代码。
14   output: {
15     // 输出文件存放的目录, 必须是 string 类型的绝对路径。
16     path: path.resolve(__dirname, 'dist'),
17
18     // 输出文件的名称
19     filename: 'bundle.js', // 完整的名称
20     filename: '[name].js', // 当配置了多个 entry 时, 通过名称模版为不同的 entry 生成不同的文件名称
21     filename: '[chunkhash].js', // 根据文件内容 hash 值生成文件名称, 用于浏览器长时间缓存文件
22
23     // 发布到线上的所有资源的 URL 前缀, string 类型
24     publicPath: '/assets/', // 放到指定目录下
25     publicPath: '', // 放到根目录下
26     publicPath: 'https://cdn.example.com/', // 放到 CDN 上去
27
28     // 导出库的名称, string 类型
29     // 不填它时, 默认输出格式是匿名的立即执行函数
30     library: 'MyLibrary',
31
32     // 导出库的类型, 枚举类型, 默认是 var
33     // 可以是 umd | umd2 | commonjs2 | commonjs | amd | this | var | assign | window | global | jsonp ,
34     libraryTarget: 'umd',
35
36     // 是否包含有用的文件路径信息到生成的代码里去, boolean 类型
37     pathinfo: true,
38
39     // 附加 Chunk 的文件名称
40     chunkFilename: '[id].js',
41     chunkFilename: '[chunkhash].js',
42

```

```

43 // JSONP 异步加载资源时的回调函数名称, 需要和服务端搭配使用
44 jsonpFunction: 'myWebpackJsonp',
45
46 // 生成的 Source Map 文件名称
47 sourceMapFilename: '[file].map',
48
49 // 浏览器开发者工具里显示的源码模块名称
50 devtoolModuleFilenameTemplate: 'webpack:///[resource-path]',
51
52 // 异步加载跨域的资源时使用的方式
53 crossOriginLoading: 'use-credentials',
54 crossOriginLoading: 'anonymous',
55 crossOriginLoading: false,
56 },
57
58 // 配置模块相关
59 module: {
60   rules: [ // 配置 Loader
61     {
62       test: /\.jsx?$/, // 正则匹配命中要使用 Loader 的文件
63       include: [ // 只会命中这里面的文件
64         path.resolve(__dirname, 'app')
65       ],
66       exclude: [ // 忽略这里面的文件
67         path.resolve(__dirname, 'app/demo-files')
68       ],
69       use: [ // 使用那些 Loader, 有先后次序, 从后往前执行
70         'style-loader', // 直接使用 Loader 的名称
71         {
72           loader: 'css-loader',
73           options: { // 给 html-loader 传一些参数
74             // ...
75           }
76         }
77       ],
78     },
79     noParse: [ // 不用解析和处理的模块
80       /special-library\.js$/ // 用正则匹配
81     ],
82   ],
83
84 // 配置插件
85 plugins: [
86   // ...
87 ],
88
89 // 配置寻找模块的规则
90 resolve: {
91   modules: [ // 寻找模块的根目录, array 类型, 默认以 node_modules 为根目录

```



```

91     'node_modules',
92     path.resolve(__dirname, 'app')
93 ],
94 extensions: ['.js', '.json', '.jsx', '.css'], // 模块的后缀名
95 alias: { // 模块别名配置, 用于映射模块
96     // 把 'module' 映射 'new-module', 同样的 'module/path/file' 也会被映射
成 'new-module/path/file'
97     'module': 'new-module',
98     // 使用结尾符号 $ 后, 把 'only-module' 映射成 'new-module',
99     // 但是不像上面的, 'module/path/file' 不会被映射成 'new-module/path/fil
e'
100     'only-module$': 'new-module',
101 },
102 alias: [ // alias 还支持使用数组来更详细的配置
103 {
104     name: 'module', // 老的模块
105     alias: 'new-module', // 新的模块
106     // 是否是只映射模块, 如果是 true 只有 'module' 会被映射, 如果是 false 'm
odule/inner/path' 也会被映射
107     onlyModule: true,
108 },
109 ],
110 symlinks: true, // 是否跟随文件软链接去搜寻模块的路径
111 descriptionFiles: ['package.json'], // 模块的描述文件
112 mainFields: ['main'], // 模块的描述文件里的描述入口的文件的字段名称
113 enforceExtension: false, // 是否强制导入语句必须要写明文件后缀
114 },
115
116 // 输出文件性能检查配置
117 performance: {
118     hints: 'warning', // 有性能问题时输出警告
119     hints: 'error', // 有性能问题时输出错误
120     hints: false, // 关闭性能检查
121     maxAssetSize: 200000, // 最大文件大小 (单位 bytes)
122     maxEntrypointSize: 400000, // 最大入口文件大小 (单位 bytes)
123     assetFilter: function(assetFilename) { // 过滤要检查的文件
124         return assetFilename.endsWith('.css') || assetFilename.endsWith('.j
s');
125     }
126 },
127
128 devtool: 'source-map', // 配置 source-map 类型
129
130 context: __dirname, // Webpack 使用的根目录, string 类型必须是绝对路径
131
132 // 配置输出代码的运行环境
133 target: 'web', // 浏览器, 默认
134 target: 'webworker', // WebWorker

```



```

135     target: 'node', // Node.js, 使用 `require` 语句加载 Chunk 代码
136     target: 'async-node', // Node.js, 异步加载 Chunk 代码
137     target: 'node-webkit', // nw.js
138     target: 'electron-main', // electron, 主线程
139     target: 'electron-renderer', // electron, 渲染线程
140
141     externals: { // 使用来自 JavaScript 运行环境提供的全局变量
142       jquery: 'jQuery'
143     },
144
145     stats: { // 控制台输出日志控制
146       assets: true,
147       colors: true,
148       errors: true,
149       errorDetails: true,
150       hash: true,
151     },
152
153     devServer: { // DevServer 相关的配置
154       proxy: { // 代理到后端服务接口
155         '/api': 'http://localhost:3000'
156       },
157       contentBase: path.join(__dirname, 'public'), // 配置 DevServer HTTP 服
务器的文件根目录
158       compress: true, // 是否开启 gzip 压缩
159       historyApiFallback: true, // 是否开发 HTML5 History API 网页
160       hot: true, // 是否开启模块热替换功能
161       https: false, // 是否开启 HTTPS 模式
162     },
163
164     profile: true, // 是否捕捉 Webpack 构建的性能信息, 用于分析什么原因导致构建性
能不佳
165
166     cache: false, // 是否启用缓存提升构建速度
167
168     watch: true, // 是否开始
169     watchOptions: { // 监听模式选项
170       // 不监听的文件或文件夹, 支持正则匹配。默认为空
171       ignored: /node_modules/,
172       // 监听到变化发生后等300ms再去执行动作, 防止文件更新太快导致重新编译频率太高
173       // 默认为300ms
174       aggregateTimeout: 300,
175       // 判断文件是否发生变化是不停的去询问系统指定文件有没有变化, 默认每隔1000毫秒询问
一次
176       poll: 1000
177     },
178   }

```

4.9. 多种配置类型

除了通过导出一个 Object 来描述 Webpack 所需的配置外，还有其它更灵活的方式，以简化不同场景的配置。

4.9.1. 导出一个 Function

在大多数时候你需要从同一份源代码中构建出多份代码，例如一份用于开发时，一份用于发布到线上。

如果采用导出一个 Object 来描述 Webpack 所需的配置的方法，需要写两个文件。一个用于开发环境，一个用于线上环境。再在启动时通过 `webpack --config webpack.config.js` 指定使用哪个配置文件。

采用导出一个 Function 的方式，能通过 JavaScript 灵活的控制配置，做到只用写一个配置文件就能完成以上要求。

导出一个 Function 的使用方式如下：

 复制代码

```
1  const path = require('path');
2  const UglifyJsPlugin = require('webpack/lib/optimize/UglifyJsPlugin');
3
4  module.exports = function (env = {}, argv) {
5    const plugins = [];
6
7    const isProduction = env['production'];
8
9    // 在生成环境才压缩
10   if (isProduction) {
11     plugins.push(
12       // 压缩输出的 JS 代码
13       new UglifyJsPlugin()
14     )
15   }
16
17   return {
18     plugins: plugins,
19     // 在生成环境不输出 Source Map
20     devtool: isProduction ? undefined : 'source-map',
21   };
22 }
```

4.9.2. 导出一个 Promise

Webpack 还支持导出一个返回 Promise 的函数，使用如下：

 复制代码

```
1 module.exports = function(env = {}, argv) {
2   return new Promise((resolve, reject) => {
3     setTimeout(() => {
4       resolve({
5         // ...
6       })
7     }, 5000)
8   })
9 }
```

4.9.3. 导出多份配置

除了只导出一份配置外，Webpack 还支持导出一个数组，数组中可以包含每份配置，并且每份配置都会执行一遍构建。使用如下：

 复制代码

```
1 module.exports = [
2   // 采用 Object 描述的一份配置
3   {
4     // ...
5   },
6   // 采用函数描述的一份配置
7   function() {
8     return {
9       // ...
10    }
11  },
12  // 采用异步函数描述的一份配置
13  function() {
14    return Promise();
15  }
16 ]
```

以上配置会导致 Webpack 针对这三份配置执行三次不同的构建。

这特别适合于用 Webpack 构建一个要上传到 Npm 仓库的库，因为库中可能需要包含多种模块化格式的代码，例如 CommonJS 、 UMD 。

4.9.10. 总结

Webpack 内置了很多功能。你不必都记住它们，只需要大概明白 Webpack 原理和核心概念去判断选项大致属于哪个大模块下，再去查详细的使用文档。

通常你可用如下经验去判断如何配置 Webpack：

- 想让源文件加入到构建流程中去被 Webpack 控制，配置 entry；
- 想自定义输出文件的位置和名称，配置 output；
- 想自定义寻找依赖模块时的策略，配置 resolve；
- 想自定义解析和转换文件的策略，配置 module，通常是配置 module.rules 里的 Loader；
- 其它的大部分需求可能要通过 Plugin 去实现，配置 plugin；